IT skills in action

Well done! You've gained a strong foundation in various crucial aspects of IT, from version control to project management. You've also practiced how to:

- Implement version control using git and Github
- Branch and merge your work
- Secure and restore repositories
- Resolve code conflicts
- Run code reviews and manage pull requests
- Use versioning to track and manage projects

Now, let's put your skills into action with a real-world scenario that encompasses the lessons you've learned.

The project

Imagine you're part of an IT team responsible for developing and managing a software project. Your team is using Git for version control, collaborating on coding tasks, and ensuring project success. Let's walk through the process step by step.

Project steps

Before Version Control: Before diving into code, ensure your team is aligned on the project's scope, goals, and responsibilities.

Version control systems: Choose Github as your version control system to track changes, collaborate effectively, and maintain a history of your project.

Using git: Start by initializing a Github repository, committing your initial code, and using git status and git log to manage and track changes.

Advanced git interaction: Use advanced commands like git diff to visualize changes, git stash to temporarily hide changes, and git tag to mark significant milestones.

Undoing things: Use git reset and git revert to undo changes and address errors in a controlled manner.

Branching and merging: Create branches for feature development using git branch, switch between branches with git checkout, and merge changes using git merge.

Secure shells & API keys: Ensure security by using SSH keys and managing sensitive data like API keys properly.

Solving conflicts: Resolve conflicts that arise from merging branches using git merge or pull requests.

Pull requests: Open pull requests to propose changes, review code, and discuss modifications with your team.

Code reviews: Participate in code reviews to maintain code quality, identify improvements, and ensure best practices.

Managing projects: Organize your project using project boards, milestones, and issues to track progress and prioritize tasks.

Putting it all together

Imagine you're assigned to add a new feature to your project: a user authentication system. Here's how you'd apply your skills:

Before version control: Working with your development team and stakeholders you define the feature's scope and priorities. From the business requirements you develop user stories from which the team can build out tasks. Review the tasks your team created and discuss expected outcomes.

Version control systems: You create a feature branch for the authentication system on the app's existing repository that is already located on github. Your team uses this new branch to begin to work on the tasks associated with the feature request.All progress is tracked in real time and documented with comments in Github.

```
# Create a new feature branch

git checkout -b feature/user-authentication
```

Advanced git interaction: You use git diff to view and compare code changes and look back at the history of changes. When needed you can use git diff to compare whole branches as the feature becomes more robust. As you get closer to completing the feature you create tags to mark development milestones. When feature release is approaching, you can use a milestone to share progress with stakeholders.

```
# View code changes

git diff

# View commit history

git log

# Create a new tag

git tag v1.0.0

# Compare branches

git diff feature/user-authentication main
```

Undoing things: As you encounter issues, you have stable milestones you know you can restore back. You can stash away pending changes or, safely undo changes using Git's commands.

```
# Stash changes

git stash

# Restore changes from stash

git stash pop

# Undo changes in working directory

git checkout -- <file>
```

Branching and merging: Your team makes sure to keep up with branching and merging changes. The team tests their changes in the feature branch to avoid introducing any issues or bugs into the main branch.

```
# Merge changes from feature branch to main

git checkout main

git merge feature/user-authentication

# Delete feature branch

git branch -d feature/user-authentication
```

Solving Conflicts: As code conflicts arise during merging, you attempt to automerge. When deeper conflicts arise, you gather your team and address them collaboratively.

```
# Attempt to automerge

git merge feature/user-authentication

# Resolve conflicts manually

# Edit files to resolve conflicts

git add <resolved-files>

git commit -m "Resolved conflicts"
```

Pull requests and code reviews: One of your team members opens up a pull request for your feature branch. It is finally time to merge our feature into the main branch. Automated tests run against the code in question and your team schedules a code review. You prepare to gather and track feedback.

```
# Push changes and open pull request

git push origin feature/user-authentication

# Automated tests run in CI/CD pipeline

# Pull request is reviewed
```

```
# Feedback is addressed
```

Code reviews: All concerned parties participate in code reviews. Team members address the group and review their code additions. Tests and metrics are also reviewed. The team collaborates at addressing feedback and ensuring high-quality code.

Managing projects: Throughout the project, and even after development efforts have concluded, you continue to track the progress of your feature using project boards, milestones, and issues. Development is iterative and your team will continue to work on features as feedback and requests come in from stakeholders.

By applying your skills across the development life-cycle, you've successfully contributed to the project's growth and demonstrated your expertise in IT and project management.

Key takeaways

Throughout this guided activity, you've delved into the practical application of various IT skills, following a step-by-step process that encapsulates the skills you've learned. You've navigated the world of version control systems, using Git's essential functionalities, branching strategies, and remote repository interactions. With a keen eye for detail, you've tackled code reviews and confidently resolved conflicts, ensuring the seamless collaboration essential for effective software development. Your journey also encompassed essential project management aspects, where you employed project boards, milestones, and issue tracking to oversee and guide your projects' evolution. Feel confident to harness your IT skills in real-world contexts, paving the way for efficient, collaborative, and successful software development endeavors.